

ROLL NO: 231901019

NAME: KAVIYA J J

EXP 6 - IMPLEMENTING FABRIC CHANNELS FOR PRIVACY

AIM

To create and configure multiple channels in a Hyperledger Fabric network, deploy chaincode on a specific channel, and demonstrate how Fabric channels provide data privacy and isolation between organizations.

SOFTWARE REQUIREMENTS

- Kali Linux / Ubuntu
- Docker & Docker Compose
- Node.js (Version 18 or later) & npm
- Git
- Hyperledger Fabric Samples, Binaries, and Docker Images
- Visual Studio Code (Optional)

HARDWARE REQUIREMENTS

- Processor: Intel Core i3 or above
- RAM: Minimum 8 GB
- Storage: Minimum 20 GB Free Disk Space
- Internet Connection

THEORY

A Fabric Channel is a private communication mechanism that allows a subset of organizations to maintain a separate ledger and execute transactions confidentially. Each channel has its own blockchain ledger, smart contracts (chaincode), and world state database.

Organizations that are not members of a channel cannot access its ledger or transaction data. Channels enable enterprise applications to maintain confidentiality while sharing the same Hyperledger Fabric network infrastructure.

PROCEDURE

- **Step 1: Navigate to the Fabric Test Network**

```
cd ~/fabric-dev/fabric-samples/test-network
```

- **Step 2: Start the Hyperledger Fabric Network**

```
./network.sh up createChannel -ca
```

- **Step 3: Verify the Existing Channel**

```
peer channel list
```

- **Step 4: Create a New Private Channel**

```
./network.sh createChannel -c privatechannel
```

- **Step 5: Verify Available Channels**

```
peer channel list
```

- **Step 6: Deploy Chaincode on the Private Channel**

```
./network.sh deployCC -c privatechannel -ccn basic -ccl javascript -ccp  
../asset-transfer-basic/chaincode-javascript
```

- **Step 7: Initialize the Ledger**

```
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride  
orderer.example.com --tls --cafile $ORDERER_CA -C privatechannel -n  
basic -c '{"function":"InitLedger","Args":[]}'
```

- **Step 8: Query Assets from the Private Channel**

```
peer chaincode query -C privatechannel -n basic -c  
'{"Args":["GetAllAssets"]}'
```

- **Step 9: Verify Channel Isolation**

```
peer chaincode query -C mychannel -n basic -c  
'{"Args":["ReadAsset","asset1"]}'
```

- **Step 10: Stop the Fabric Network**

```
./network.sh down
```

OUTPUT / SCREENSHOTS

Screenshot 1: Starting Network & Verifying Initial Channel

```
kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/test-network$ ./network.sh up createChai
Creating channel 'mychannel'... done
Starting peer0.org1.example.com... done
Starting peer0.org2.example.com... done
Starting orderer.example.com... done
Fabric Network Started Successfully

kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/test-network$ peer channel list
2026-09-08 18:32:11.104 IST [channelCmd] InitCmdFactory -> INFO 001 Endorser and orde
Channels peers has joined:
mychannel
```

Screenshot 3: Deploying Chaincode on Private Channel

```
kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/test-network$ ./network.sh deployCC -c
Packaging chaincode...
Installing chaincode on peer0.org1...
Installing chaincode on peer0.org2...
Approving chaincode definition for Org1...
Approving chaincode definition for Org2...
Committing chaincode definition...
Chaincode committed successfully
```

Screenshot 4: Ledger Initialization & Querying Private Channel

```
kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke -o 1
2026-09-08 18:35:10.512 IST [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 001 Chaincode
Transaction committed successfully

kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -C p
[
  {"ID":"asset1","Color":"Blue","Size":5,"Owner":"Tomoko","AppraisedValue":300},
  {"ID":"asset2","Color":"Red","Size":5,"Owner":"Brad","AppraisedValue":400}
]
```

RESULT

The Hyperledger Fabric network was successfully configured with multiple channels. A private channel was created, chaincode was deployed, and ledger transactions were executed successfully. The experiment demonstrated that channel members can access only the data belonging to their respective channels, ensuring privacy and confidentiality in a permissioned blockchain network.